

Game Flow Documentation

Overview

This document explains the complete game flow between the **Shunya App**, the **Backend APIs**, and the **Game Provider**.

The `key` is generated using the following format:

```
key = contestId::playerId
```

Example:

```
eae5816f-f3d0-461e-8030-c10ee810c170::104
```

Where:

- `contestId` = `eae5816f-f3d0-461e-8030-c10ee810c170`
 - `playerId` = `104`
-

1. Start Game

The game is initiated from the **Shunya App**.

The app calls the `startGame` API to create a game session for the player.

API Request

```
curl --location 'http://localhost:8088/UserContest/eae5816f-f3d0-461e-8030-c10ee810c170' \
--header 'Authorization: Bearer <token>' \
--header 'Content-Type: application/json' \
--data '{
  "providerId": "a0a08d9d-1055-4ef4-bc9c-a3605f3c8467",
  "playerId": 104,
  "contestId": "eae5816f-f3d0-461e-8030-c10ee810c170"
}
```

```
"stageId": "41861ce7-f86c-441e-b8f0-814999afe61a"
}'
```

Request Details

Path Parameter

Parameter	Description
contestId::playerId	Unique key generated using contest ID and player ID

Example:

```
eae5816f-f3d0-461e-8030-c10ee810c170::104
```

Headers

Header	Description
Authorization	Bearer token of the authenticated user
Content-Type	Request payload type

Request Body

Field	Description
providerId	Unique ID of the game provider
stageId	Unique ID of the game stage

API Response

```
{
  "gameSessionId": "fdc9055c-5ee0-44ba-91ae-bae726891ea9",
  "gameUrl": "https://game-provider/mcq",
}
```

Response Details

Field	Description
<code>gameSessionId</code>	Unique session ID generated for the game
<code>gameUrl</code>	URL that should be opened inside the WebView

2. Open Game in WebView

After receiving the response from the `startGame` API:

1. The **Shunya App** opens the `gameUrl` inside a WebView.
2. The app injects the following values into the game:
 - `token`
 - `gameSessionId`
 - `key`

These values are required by the Game Provider to authenticate and identify the active player session.

3. Submit Game Result

Once gameplay is completed, the **Game Provider** submits the result using the `submitGameResult` API.

API Request

```
curl --location 'http://localhost:8088/UserContest/eae5816f-f3d0-461e-803' \
--header 'Content-Type: application/json' \
--data '{
  "gameSessionId": "58e6b3a0-f822-43bb-b97e-1a78377bd1f4",
  "outcome": "completed",
  "score": 100
}'
```

Request Details

Headers

Header	Description
Authorization	Bearer token of the authenticated user
Content-Type	Request payload type

Request Body

Field	Description
gameSessionId	Session ID received from the <code>startGame</code> API
outcome	Final game status (example: <code>completed</code>)
score	Final score achieved by the player

API Response

```
{
  "success": true
}
```

4. Get Game Session State

The `getGameSessionState` API can be used to fetch the current state/details of a game session.

This API helps retrieve metadata related to the session, such as when the game was started.

API Request

```
curl --location 'http://localhost:8088/UserContest/eae5816f-f3d0-461e-803' \
--header 'Content-Type: application/json' \
--data '{
```

```
"gameSessionId": "892d85f0-2e05-4731-8b6f-adbd358de8db"
}'
```

Request Details

Request Body

Field	Description
gameSessionId	Unique game session ID

API Response

```
{
  "gameSessionId": "892d85f0-2e05-4731-8b6f-adbd358de8db",
  "gameStartedAt": "2026-05-22T09:20:52.821Z"
}
```

Response Details

Field	Description
gameSessionId	Unique session ID of the game
gameStartedAt	Timestamp when the game session started

Complete Flow Summary





Important Notes

- `gameSessionId` is unique for every game session.
- `key` uniquely identifies the contest-player combination.
- Authentication is handled using the `Authorization: Bearer <token>` header.
- The Game Provider should always use the same `gameSessionId` received during game initialization.
- The `submitGameResult` API should only be called after gameplay completion.
- The injected `token`, `gameSessionId`, and `key` should be securely handled inside the WebView/game environment.